RouteStage

Technical Report — Optimizing Shipment Routes Under Cost, Time, Reliability, and Risk Constraints

Executive Summary

Logistics planners must balance conflicting priorities—minimizing cost and transit time while maximizing reliability and mitigating cargo risk. Conventional route selection tools either rely on static heuristic rules or obscure decision logic behind unexplainable "black-box" predictions.

RouteStage is an explainable, multi-objective decision-support platform designed to evaluate routing candidates, recommend optimal paths tailored to user-defined operational trade-offs, and generate verifiable natural-language explanations. The platform is governed by a core architectural principle: **"The optimizer decides. The LLM explains."** All constraint evaluations, metric normalizations, utility scoring, ranking, Pareto trade-off identification, and sensitivity analyses are performed by a strictly deterministic mathematical engine. Google Gemini (via the official `google-genai` SDK) serves purely as an explanation and presentation layer, converting structured mathematical evidence into natural language under strict schema constraints and grounding validation.

1. Problem Definition

Freight transportation involves multi-criteria decisions under operational uncertainty. For any shipment defined by origin, destination, cargo classification, weight, commercial value, and mandatory delivery deadline, numerous candidate paths exist across air, ocean, rail, and road. Planners must balance four competing dimensions: 1. **Financial Expenditure (Total Cost):** Direct freight, handling, and tariff charges (lower is better). 2. **Operational Velocity (Transit Time):** End-to-end transport duration in hours (lower is better). 3. **Delivery Predictability (Reliability):** Historical on-time arrival rate without disruption (higher is better). 4. **Vulnerability (Aggregate Risk):** Exposure to damage, delay, weather, and geopolitical factors (lower is better).

2. Definition of Optimality

An **optimal route** is defined as:

> *The feasible candidate route that achieves the highest normalized weighted score under the user's selected Optimization Profile. Pareto analysis is used separately to identify non-dominated alternatives and expose operational trade-offs.*

Under this definition: - **Feasibility precedes scoring:** A candidate violating hard operational constraints has an undefined utility score and can never be recommended. - **Optimality is preference-dependent:** A route optimal under a "Cost-First" profile may be suboptimal under a "Time-Critical" profile. - **Separation of ranking and trade-offs:** Pareto analysis identifies non-dominated alternatives to present viable trade-offs to human planners; Pareto efficiency does not imply global optimality across all possible user preferences.

3. Decision Framework / Architecture

The system decouples mathematical optimization from narrative explanation:

- **Deterministic Optimization Engine:** The sole authority for evaluating hard constraints, normalizing metrics, calculating weighted utility, ranking candidates, generating recommendations, computing the Pareto frontier, and running sensitivity analysis. - **Evidence Snapshot:** An immutable snapshot (`ExplanationEvidence`) captures verified facts directly from the optimizer result. - **Explanation Layer:** Google Gemini generates a structured JSON explanation based strictly on the snapshot. - **Grounding Safeguard:** Output is validated against optimizer decisions; any hallucination or contradiction triggers a deterministic fallback.

The LLM has zero authority to select routes, alter scores, modify weights, or change feasibility classifications.

4. Hard Constraints and Feasibility

Before scoring, the `ConstraintEngine` evaluates candidates against hard operational boundaries: 1. **Delivery Deadline:** Total transit time must not exceed the shipment's required arrival window. 2. **Cost-to-Value Limit:** Total cost must not exceed the maximum allowable budget relative to cargo value. 3. **Maximum Risk Tolerance:** Aggregate risk must remain below the threshold defined for the cargo and priority.

Violating candidates are marked infeasible with specific violation reasons and excluded from scoring. If all candidates are infeasible, the engine cleanly returns zero recommendations (`recommended_route_id = None`) rather than selecting an invalid route.

5. Multi-Objective Scoring

Feasible candidates are scored across the four dimensions (C: Cost, T: Time, R: Reliability, K: Risk).

Directional Min-Max Normalization Metrics are scaled to $[0.0, 1.0]$, where 1.0 represents the best performance: - **Lower is Better (Cost, Time, Risk):** $S_{i, d} = (\max_j x_{j, d} - x_{i, d}) / (\max_j x_{j, d} - \min_j x_{j, d} + \epsilon)$ - **Higher is Better (Reliability):** $S_{i, d} = (x_{i, d} - \min_j x_{j, d}) / (\max_j x_{j, d} - \min_j x_{j, d} + \epsilon)$

Weighted Utility Calculation The user specifies an `OptimizationProfile` where weights strictly sum to 1.0:

$$\text{Total Score}_i = w_{\text{cost}} \cdot S_{i, \text{cost}} + w_{\text{time}} \cdot S_{i, \text{time}} + w_{\text{reliability}} \cdot S_{i, \text{reliability}} + w_{\text{risk}} \cdot S_{i, \text{risk}}$$

Ties are broken deterministically by: (1) Lowest Cost, (2) Lowest Transit Time, and (3) Stable route UUID.

6. Pareto Analysis

The `ParetoAnalyzer` identifies the non-dominated set among feasible routes. Route A dominates Route B ($A \succ B$) if A is at least as good as B in all four dimensions and strictly superior in at least one. Non-dominated routes form the **Pareto Frontier**. This provides planners with mathematically sound alternatives that represent legitimate trade-offs under different priority balances.

7. Sensitivity Analysis

To assess recommendation stability, the `SensitivityAnalyzer` perturbs the user's objective weights by **$\pm 20\%$** (`deltas = [0.2, -0.2]`). For each dimension, the focused weight is shifted by ± 0.20 (bounded within $[0.0, 1.0]$) while other weights are re-scaled proportionally to sum to 1.0.

A **Stability Score** is computed as the percentage of perturbation scenarios where the baseline recommendation remains the winner. If minor weight changes flip the winner, the recommendation is flagged as "Sensitive", alerting the operator to fragile trade-off margins.

8. Explanation Architecture

1. **Evidence Builder:** Extracts verified data from the frozen `OptimizationResult` into `ExplanationEvidence`. 2. **Constrained Generation:** Gemini 2.5 Flash receives the evidence with strict instructions: "*The optimizer decides. You only explain.*" 3. **Structured JSON Schema:** Gemini must return a typed `StructuredExplanation` object (`recommended_route_name`, `reasons[]`, `tradeoffs[]`, `constraint_notes[]`). 4. **Grounding Validation:** `ExplanationGenerator._validate_structured` verifies: - Recommended route name exactly matches the optimizer's choice. - No route is recommended if no feasible route exists. - Infeasible routes are never described as feasible or selectable. 5. **Deterministic Fallback:** If the API key is missing, network calls fail or time out, or grounding validation fails, the platform automatically returns a verified deterministic explanation template.

Limitation: While grounding guarantees that the LLM cannot alter recommendations or feasibility, numerical claims within free-text `reasons` strings are not exhaustively audited by NLP extraction. However, such claims cannot alter routing decisions.

9. Dataset

The system includes a reproducible synthetic dataset generator (`SyntheticDataGenerator`, fixed `seed=42`): - **30 Shipments:** Diverse global origins and destinations across 10 major commercial hubs, covering multiple cargo types (General, Perishable, Hazardous), weights (100–50,000 kg), values (\$5,000–\$1,000,000), and deadlines (7–45 days). - **120 Candidate Routes (4 per shipment):** Balanced, Economy (low cost, slower), Express (fast, premium cost), and High-Risk Low-Cost profiles, composed of 1–3 multimodal transport legs (Air, Ocean, Rail, Road) with consistent transit metrics.

10. Evaluation Methodology

Rather than measuring fictitious "prediction accuracy" (as there is no arbitrary ground-truth route in multi-objective optimization), the decision engine is evaluated against operational and mathematical invariants: 1. **Zero-Infeasible Guarantee:** Infeasible routes must never be recommended. 2. **All-Infeasible Grace:** Scenarios with zero feasible routes must report no recommendation without error. 3. **Pareto Alignment:** Recommendations should be non-dominated within candidate sets under standard weights. 4. **Deterministic Repeatability:** Identical inputs must produce identical outputs across repeated runs.

Evaluation is executed programmatically via `backend/scripts/evaluate_decision_system.py`.

11. Evaluation Results

The evaluation script produced the following verified metrics across the synthetic dataset:

Evaluation Metric	Measured Value	Requirement / Benchmark	Status
Total Shipments Evaluated	30	30	**PASS**
Total Candidate Routes	120	120 (4 per shipment)	**PASS**
Shipments with Feasible Route	29	Expected feasible cases	**PASS**
Shipments with No Feasible Route	1	Handled cleanly (0 recommended)	**PASS**
Total Recommendations Made	29	Exactly matches feasible shipments	**PASS**
Infeasible Routes Recommended	**0**	**0 (Strict Safety Invariant)**	**PASS**
Average Recommended Cost	**$8,120.66**	Within reasonable budget bounds	**PASS**
Average Recommended Transit Time	**13.22 hours**	Well within delivery deadlines	**PASS**
Average Recommended Reliability	**88.83%**	High on-time predictability	**PASS**
Average Recommended Risk Index	**0.1400**	Low vulnerability profile (< 0.25)	**PASS**
Pareto-Efficient Recommendations	**29 / 29 (100%)**	Non-dominated in candidate sets	**PASS**
Deterministic Repeatability	**PASS**	100% repeatable across runs	**PASS**

Note on Pareto Metric: The 100% Pareto efficiency confirms that for all 29 recommendations, no candidate in the evaluated set strictly dominated the chosen route. It does not imply universal optimality across all hypothetical preferences.

12. Testing and Reliability

Full-stack automated verification confirms systemic stability: - **Backend Test Suite (`pytest`): 81 / 81 tests passed** covering constraints, normalization, scoring, tie-breaking, Pareto sorting, sensitivity, persistence, grounding, and fallbacks. - **Frontend Test Suite (`Jest`): 18 / 18 tests passed** covering shipment workflows, route evaluation UI, and route-leg rendering. - **Type Checking & Linting:** `npx tsc --noEmit` passed with **0 errors**; Next.js ESLint passed with **0 errors / 0 warnings**. - **Production Build:** Next.js 14 compiled and generated all static/dynamic routes successfully. - **Security Audit:** Zero secrets or API keys committed; `.env` is gitignored; `.env.example` contains only placeholders.

13. Limitations and Future Work

1. **In-Memory Candidate Evaluation:** Candidate routes are evaluated in memory. For enterprise-scale routing graphs with millions of combinations, an upstream graph search or mixed-integer programming (MIP) formulation is required to generate candidate paths prior to scoring. 2. **Predefined Perturbation Grid:** Sensitivity analysis uses discrete $\pm 20\%$ perturbations across primary axes rather than calculating full continuous indifference regions. 3. **LLM Numerical Auditing:** Grounding guarantees recommendation integrity, but free-text numerical claims within narrative explanations are not exhaustively parsed against source metrics. 4. **Synthetic Data Realism:** The dataset validates algorithmic behavior reliably, but commercial production requires integration with live telematics, carrier EDI, port congestion indices, and real-time weather APIs.

14. Conclusion

RouteStage demonstrates an effective, responsible architecture for logistics decision support. By strictly separating deterministic optimization from generative explanation, the platform delivers mathematically sound, repeatable route recommendations while providing operators with clear, grounded visibility into operational trade-offs.